SCIENTIFIC CLAIM AUDIT / REVIEWER EDITION

Find the weakness in your paper before reviewers do

An academic paper can look convincing while hiding problems across its manuscript, data, code, supplementary materials, or preregistration. Finding those problems after submission can mean major revisions, rejection, or months of avoidable work. ClaimBounty gives you an independent scientific audit before the paper reaches reviewers.

REVIEWER PATH
Install → Import → Run → Inspect
Toone scientific workflow

AUTHORSHIP
Authored by Matheus Paranhos
Public hackathon repository projection

FIVE-MINUTE REVIEWER PATH

Install the workflow, run the example, inspect the evidence.

ClaimBounty's scientific audit runs locally in Toone. This guide takes a reviewer from a clean installation through workflow import, one included scientific case, the agent trajectory, and the final decision package.

00:00 **Install the local runtime**

Install Toone, one supported coding-agent client, and R for the included Heliconius example.

01:00 **Import ClaimBounty**

Create an empty Toone project, open *Explore Workflows*, select `micro1/ClaimBounty`, and import the complete workflow.

02:00 **Run the included case**

Copy the Moura et al. Heliconius Experiment 1 bundle, bind the case and policy inputs, and start the parent workflow.

03:00 **Follow the agents**

Inspect each child routine, tool response, evidence handoff, continuation checkpoint, independent review, and adjudication record.

04:00 **Review the decision package**

Confirm the verdict, robustness summary, consistency findings, prioritized corrections, reproduction evidence, provenance, and review controls.

CHOOSE THE RIGHT PATH

Four modes, four different claims.

MODE	PREREQUISITES	EXPECTED RESULT	DOES NOT PROVE
Visual preview	Node 22 from `.node-version`; pnpm 9.15.0	Static interface at the Vite preview URL, normally `http://localhost:4173`	API behavior, upload, email, storage, scanning, worker, or export
Hosted stack	Git, Docker Engine, Docker Compose v2, free local ports, local `.env`	Web at `http://127.0.0.1:8081`; mail at `http://localhost:8025`	Scientific execution, adjudication, payment, or publication
Scientific workflow	macOS, latest native Toone, one supported coding-agent client (Codex or Claude Code), Python 3.11+, study runtimes, verified case	Local audit artifacts under project-bound paths and a reviewer package	Release verification of the Explore Workflows listing or benchmark qualification
Contributor gates	Hosted prerequisites plus Go 1.26.6; more disk and time for full gates	Contract, architecture, unit, browser, system, package, and release checks as selected	Production readiness without secrets, TLS, identity, lifecycle, monitoring, and policy review

Visual-only preview

```
pnpm install --frozen-lockfile
pnpm --filter @micro1/web build
pnpm --filter @micro1/web preview
```

Expected terminal output includes the local Vite URL. API-backed actions will not complete in this mode.

Hosted stack

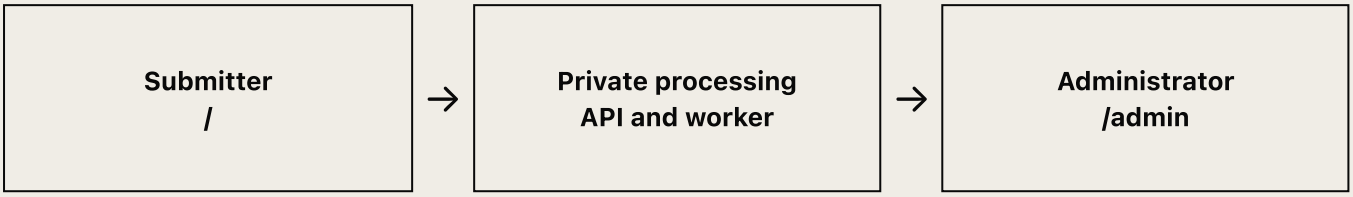
```
cp .env.example .env
docker compose --project-name claimbounty-public \
  --file infra/compose.yaml --profile claimbounty \
  up --build -d --wait
```

Replace demonstration credentials before any exposure beyond the local machine. Use synthetic data for public demonstrations.

Stop without deleting volumes: `docker compose --project-name claimbounty-public --file infra/compose.yaml --profile claimbounty down`

SUBMITTER AND ADMINISTRATOR PATH

From private upload to authorized export.



SUBMITTER

1. Select paper and materials

Open `http://127.0.0.1:8081`. Add exactly one PDF marked **Primary paper**, then optional documents, data, code, notebooks, or a ZIP archive. The interface accepts up to 20 files, 250 MB each, and 1 GB total. The primary PDF is limited to 50 MB.

SUBMITTER

2. Verify email

Request the six-digit code, open the local mail sandbox at `http://localhost:8025`, and enter the code within ten minutes. The session is held in a secure browser cookie and not browser storage.

SUBMITTER

3. Define and authorize

Enter the study title, review purpose, exact target claim, and claim location. Record code-execution, external-search, participant-data, and direct-identifier choices. Uploads and analysis use require explicit authorization before submission.

SYSTEM

4. Inspect privately

Files enter private malware inspection. The receipt exposes a private reference and file count. The website does not open research code or dispatch a scientific audit.

ADMINISTRATOR

5. Review and freeze intake

Sign in at `http://127.0.0.1:8081/admin` with the configured allowlisted email. Review metadata, privacy declarations, file scan states, and the server-owned retention policy. Freeze the local handoff intake.

ADMINISTRATOR

6. Create and download export

When every included file is clean, readiness checks pass, and the trusted routine revision is current, create the immutable export. Download the ZIP and retain the displayed SHA-256 and RFC 9530 `Content-Digest` receipt.

DATA BOUNDARY

The transfer is an artifact, not an execution bridge.

HOSTED BOUNDARY

Private intake and policy

1. Browser and web application
2. API and PostgreSQL state
3. Private object storage
4. Scanner and worker
5. Administrative authorization

LOCAL BOUNDARY

Scientific execution and review

1. Whole-archive digest verification
2. Local Toone workflow
3. Study-specific runtimes
4. Independent verification
5. Audit package assembly

AUTHORIZED IMMUTABLE ZIP + TRUSTED SHA-256 + CONTENT-DIGEST → VERIFY BEFORE PARSING

Equivalent of the repository architecture diagram. Hosted credentials cannot dispatch routines. Local execution requires a separate operator decision after verification.

Hosted protections

- Server-side owner and administrator authorization on every resource read.
- Private buckets and random object keys with no email or filename.
- Bounded upload sizes, file validation, malware scanning, and fail-closed export readiness.
- Frozen source and personal-data deletion deadlines that administrators may shorten, not extend.

Public projection exclusions

- Customer data and private submissions.
- Hidden answer keys and restricted screenshots.
- Internal organization configuration and absolute workstation paths.
- The prior assessment bundle and benchmark files not cleared for redistribution.

NATIVE MACOS PATH

Install, import, select, run.

Scientific execution is local. Use the latest Toone release on macOS with one coding-agent client, Python, and the study runtime. The included Heliconius example uses R. The workflow export installs the parent and child workflows, schemas, templates, and local runner.

- ☐ **Install the latest Toone release.** Download the native macOS application from trytoone.com.
- ☐ **Install one supported coding-agent client.** Choose Codex or Claude Code, then complete that client's official sign-in flow. The next page gives the two official setup paths. Keep provider credentials outside the repository.
- ☐ **Install R.** Install the current R runtime from r-project.org. The included author script uses `lme4`, `DHARMa`, and `car`.
- ☐ **Create a project.** Open Toone, create a new project, and select an empty directory for it.
- ☐ **Add the included example.** Create `claimbounty/input/case-bundle/` and copy the contents of `examples/moura-2023-heliconius-exp1/` into it. The bundle is **Moura et al. 2023, Heliconius Experiment 1** and includes the paper, R script, data, attribution, and CC BY 4.0 license.
- ☐ **Import ClaimBounty.** Open *Explore Workflows*, select `micro1/ClaimBounty`, and import it. The release export should install every packaged workflow and resource.
- ☐ **Bind the case.** Select `claimbounty/input/case-bundle/` for the `case-bundle` input, supply the remaining dispatch documents, and confirm every study-specific runtime.
- ☐ **Run the parent workflow.** Review the bound inputs, start ClaimBounty, and treat all supplied research code as untrusted. If the release listing is unavailable, follow the repository fallback in `docs/REPRODUCE.md`.

Package verification

```
node scripts/generate-public-manifests.mjs
--check
(cd workflow/claimbounty-scientific-audit
&& \
  shasum -a 256 -c MANIFEST.sha256)
```

Expected local sequence

Freeze study case → reproduce original result → research evidence → stress-test analysis → verify and adjudicate → assemble final audit.

Source record: article DOI `10.1016/j.cub.2023.06.009`; data and code DOI `10.5281/zenodo.7985236`. File hashes and attribution are recorded in the example bundle README.

CHOOSE ONE SUPPORTED CLIENT ON MACOS

Install Codex or Claude Code before the local run.

The local scientific workflow requires Toone plus one supported coding-agent client. Codex and Claude Code are alternative client choices; install and authenticate one before importing and running the packaged workflow.

OFFICIAL INSTALL COMMAND SUPPLIED FOR THIS GUIDE

```
curl -fsSL https://chatgpt.com/codex/install.sh | sh
```

```
codex
```

Run `codex`, then choose **Sign in with ChatGPT**. Install and sign-in details can change, so use the current official Codex CLI documentation if the prompt differs.

Claude Code alternative

Follow Anthropic's official Claude Code setup guide, then complete its documented sign-in flow. This guide does not substitute an inferred or third-party installation command.

R runtime for the example

Install R, then install `lme4`, `DHARMa`, and `car`. The clean reproduction commands are in `docs/REPRODUCE.md`. With those packages installed, the checked R 4.6.1 source run completed in 1.41 seconds.

Repository inspection targets

PATH	REVIEW QUESTION
`contracts/openapi.yaml`	Does the HTTP contract preserve identity, upload, authorization, export, and digest boundaries?
`workflow/claimbounty-scientific-audit/`	Are the parent routine, six child routines, resources, runner, and manifests complete?
`submission/evidence/`	Do public claims resolve to machine-readable status, timing, and usage records?
`docs/`	Do setup, architecture, workflow, disclosure, and output-contract statements agree?

> A terminal assistant can read or run checks only with the permissions granted on the reviewer's machine. Do not expose private evidence, credentials, hidden answers, or customer files to any provider without authorization.

REPOSITORY ROOT

Run only the gate that answers the review question.

GOAL	COMMAND	EXPECTED OUTPUT
Run the included source analysis	`cd examples/moura-2023-heliconius-exp1 && Rscript exp1.R`	Model summaries, analysis-of-deviance tables, and R plot output.
Verify public package integrity	`node scripts/generate-public-manifests.mjs --check`	Exit 0; example, recording, reviewer, evidence, and workflow manifests match.
Check public links	`scripts/check-public-links.mjs`	Exit 0; repository-relative documentation links resolve.
Run public release policy	`scripts/check-public-release`	Exit 0; no prohibited private paths, risky payloads, disclosure drift, or manifest drift.
Fast contributor loop	`make check-fast`	Contract, architecture, backend, frontend, and review-hook checks pass.
Public package gate	`make public-release`	Contract and architecture registries plus public projection policy pass.
End-to-end hosted behavior	`make test-system`	Docker profile becomes healthy; browser exercises web, email, API, storage, scanner, worker, and export verifier.
Full contributor gate	`make check-ci`	Local checks, integration, vulnerability, container build, and system suite pass.

Exact local prerequisites

- Node 22; package engine minimum 20.19.0
- pnpm 9.15.0
- Go 1.26.6 for contributor gates
- Docker Engine and Compose v2 for hosted and system paths
- Python 3.11+ for the packaged local runner
- R 4.x plus lme4, DHARMa, and car for the included example

Expected repository artifacts

- `submission/reviewer/manifest.json`
- `submission/evidence/*.json`
- `workflow/claimbounty-scientific-audit/MANIFEST.sha256`
- `verified-exports/claimbounty-export/` after offline export verification

Full gates may download containers, consume substantial disk, and take longer. A visual review does not require them. Production deployment is outside this guide's verified scope.

WHAT A COMPLETE RUN SHOULD DELIVER

A decision package with evidence attached.

The final package must let a reviewer distinguish the reported result, reproduced result, defensible alternatives, unresolved conflicts, and the record of how each conclusion was reached.

Verdict

The bounded conclusion for the target claim, with status, confidence limits, and the exact evidence that supports it.

Robustness summary

How the conclusion changes across defensible data, model, covariate, and inference choices, including sensitivity limits.

Consistency findings

Agreement or conflict across paper, data, code, figures, tables, citations, and reported values.

Prioritized corrections

Actionable changes ordered by consequence, evidence, and effort, without rewriting the author's scientific judgment.

Reproduction evidence

Commands, environments, exit states, logs, numeric comparisons, tolerances, and artifact hashes needed to inspect or rerun the result.

Provenance

Source identifiers, retrieval facts, file digests, routine revisions, model usage, reviewer actions, and adjudication history.

Quality controls

- Separate technical repair from scientific comparison status.
- Preserve every material result and reviewer decision.
- Resolve public claims to released evidence.
- Keep hidden answer keys outside routines and public packages.

Review controls

- Independent method and source checks.
- Blind review where the protocol requires it.
- Adjudication before final publication.
- Evidence sealing before reference comparison.

This list is the target package contract. Every conclusion should resolve to its source, method, generated artifact, reviewer action, and adjudication record.

WHAT TO OBSERVE IN TOONE

Follow one claim from frozen inputs to adjudicated output.

Open the parent run and inspect each handoff in order. The reviewer should be able to see which agent acted, which inputs it received, which tools it used, which artifacts it produced, and which review decision allowed the workflow to continue.

01 **Confirm the frozen case**
Match the exact target claim, source location, permissions, policies, and file hashes.

02 **Inspect reproduction**
Open the recorded commands, environment, exit states, numerical comparisons, and tolerances.

03 **Inspect evidence and robustness**
Trace source checks and defensible analytical alternatives back to their inputs and outputs.

04 **Inspect independent review**
Compare the method review, source review, verification notes, and adjudication decisions.

05 **Open the final package**
Resolve every displayed conclusion to the canonical audit JSON and evidence-linked artifacts.

> In Toone, the parent run is the review spine. Child routines remain visible as evidence-producing stages, while checkpoints and handoffs explain why the workflow moved forward.

ONE PARENT, SIX EVIDENCE-PRODUCING ROUTINES

Each stage has a bounded scientific responsibility.

The parent routine coordinates the audit and preserves a single chain of evidence. Each child routine receives explicit inputs, produces named artifacts, and hands its result to the next stage through a reviewable checkpoint.

ROUTINE	RESPONSIBILITY	REVIEWER FOCUS
Build and freeze study case	Inventory and content-address the authorized inputs.	Claim identity, permissions, and hashes.
Reproduce original result	Rebuild the environment and compare the reported result.	Commands, versions, tolerances, and outputs.
Research methods and evidence	Check sources and method claims within the authorized scope.	Source quality, relevance, and traceability.
Stress-test analysis	Evaluate defensible data, model, covariate, and inference choices.	Decision space and robustness boundaries.
Verify and adjudicate findings	Independently check methods, sources, and proposed conclusions.	Reviewer independence and adjudication history.
Assemble final audit	Render the canonical audit, human report, and replay package.	JSON-to-report consistency and evidence links.

Inspect in Toone

- Agent instructions and bound inputs.
- Ordered steps and tool responses.
- Produced artifacts and handoffs.
- Human checkpoints and adjudication.

Inspect in the repository

`workflow/claimbounty-scientific-audit/routines/`

`workflow/claimbounty-scientific-audit/trajectories/`

`workflow/claimbounty-scientific-audit/project-template/`

CANONICAL ANSWERS, EVIDENCE ATTACHED

The human report and machine record must tell the same story.

Open the researcher-facing HTML or PDF beside the canonical audit JSON. Numbers, statuses, conclusions, and evidence references should agree exactly while the report keeps the ClaimBounty black, cream, Georgia, and Inter design language.

REVIEWER QUESTION	HUMAN-FACING ANSWER	EVIDENCE RECORD
What is the bounded conclusion?	Verdict with status and confidence.	Canonical claim status and linked findings.
Was the reported result reproduced?	Reported-versus-reproduced comparison.	Commands, environment, outputs, tolerances, and hashes.
How robust is the conclusion?	Robustness summary across defensible choices.	Decision-space register and sensitivity artifacts.
Do paper, data, and code agree?	Consistency findings and source locations.	Source links, checks, and independent verification.
What should the author change?	Prioritized corrections by consequence and effort.	Adjudicated recommendations and evidence links.
How was each answer reached?	Concise provenance and replay summary.	Agent actions, reviewer decisions, manifests, and digests.

Human-facing package

- Verdict and confidence.
- Robustness and consistency summary.
- Prioritized corrections.
- Evidence and provenance links.

Machine-facing package

- Canonical audit JSON.
- Commands, logs, and environments.
- Input and output hashes.
- Review and adjudication history.

Review the report as a decision interface: concise enough for an author, precise enough for an independent reviewer, and fully resolvable to the machine package.

EVERYTHING A REVIEWER NEEDS

Code, evidence, reproduction, video, and trajectories.

Solution and reproduction

- `README.md` introduces the user, bottleneck, value, and hot take.
- `IMPROVEMENT_CHANGELOG.md` connects each iteration to evidence and the next decision.
- `docs/REPRODUCE.md` gives clean setup, exact commands, versions, output, runtime, and cost.
- `examples/moura-2023-heliconius-exp1/` contains the CC BY 4.0 demonstration case.

Video and agent work

- `docs/VIDEO_SCRIPT.md` is a 4:50 shot list covering the problem, workflow execution, decision package, changelog, largest change, and removed experiment.
- `workflow/claimbounty-scientific-audit/trajectories/` contains seven routine records and nineteen agent records.
- Each trajectory records instructions, ordered steps, tool responses, feedback, retries, and human checkpoints.
- `submission/recordings/claimbounty-submission-video.mp4` is the screened 4:54 public solution video; the same folder retains its source-footage inventory.

Production monorepo

The public landing page accepts an academic bundle and one exact claim. The administrator page reviews private submissions, freezes an authorized handoff, and downloads the bundle for local Toone access. The Go backend handles identity, upload validation, private object storage, malware inspection, retention policy, immutable export creation, and worker jobs.

Public links

Toone download / native macOS workflow application

Official Codex CLI documentation

Official Claude Code setup documentation

R Project runtime

Selected benchmark data and code record

> Run `make public-release` to check the guide, links, public assets, trajectories, evidence records, and deterministic manifests together.